Day -1

WellBot – Global Wellness Assistant Chatbot

WellBot – Global Wellness Assistant Chatbot is an AI-powered conversational system designed to support users in maintaining their well-being.The chatbot interacts with users through natural language to understand their concerns and provide personalized wellness guidance, such as stress management tips, healthy lifestyle suggestions, mindfulness practices, and emotional support.

Day - 2

How to Create a Repository on GitHub (Online)

1. Open GitHub and log in to your account.
2. Click on the '+' icon in the top-right corner and select 'New repository'.
3. Enter the repository name.
4. Choose Public or Private visibility.
5. Optionally add a README file.
6. Click on 'Create repository' to finish.

How to Create a Repository using Git (Local System)

7. Open terminal or command prompt.
8. Navigate to your project folder using cd command.
9. Run 'git init' to initialize a new Git repository.
10. Add files using 'git add .'.
11. Commit changes using 'git commit -m "initial commit"'.

Day – 3

What is Git / GitHub / GitLab

Git is a version control system used to track changes in source code locally.

GitHub is a cloud platform that hosts Git repositories and enables collaboration and code sharing.

GitLab is an all-in-one DevOps platform providing Git hosting with built-in CI/CD and project management.

Git Commands (Single-Line)

git add . → Stages all modified and new files in the current directory for commit.

git commit -m "message" → Saves staged changes as a new commit with a descriptive message.

git push → Uploads local commits to the remote repository.

git pull → Fetches changes from the remote repository and merges them into the current branch.

git branch → Displays all local branches.

git branch --all → Displays both local and remote branches.

git fetch --all → Downloads updates from all remotes without merging them.

git stash → Temporarily saves uncommitted changes.

git checkout -b newbranchname → Creates a new branch and switches to it immediately.

git checkout branchname → Switches the working directory to an existing branch.

Python Virtual Environment Commands

python -m venv venv → Creates a Python virtual environment named venv.

python -m venv shakthi → Creates a Python virtual environment named shakthi.

shakthi\Scripts\activate → Activates the shakthi virtual environment on Windows.

deactivate → Deactivates the currently active virtual environment

Day 4 – SDLC Roles, Documents, Models & Agile

SDLC Roles

Functional Consultant / Business Analyst: Gathers business requirements and prepares the Functional Specification Document.

Developer / Programmer: Develops the application and performs unit testing on individual modules.

Tester / QA Team: Tests the application to ensure quality and prepares test scripts and test results.

Administration Team: Manages servers, deployment, access control, and system infrastructure.

Project / Product Manager: Plans, monitors, and controls the project to ensure timely delivery.

Sales and Marketing Team: Promotes the product and gathers customer feedback.

Human Resources: Manages recruitment, employee relations, and organizational policies.

Documentation Team: Prepares and maintains functional, technical, and user documents.

Support Team: Provides post-production support and resolves user issues.

SDLC Environments

Development Environment: Used by developers to write and test code.

Quality (QA) Environment: Used by testers to validate the application.

Production Environment: Live environment used by end users.

SDLC Documents

Functional Specification Document (FSD): Describes business requirements of the system.

Technical Specification Document (TSD): Describes technical design and implementation details.

Test Script: Defines steps to test system functionality.

Test Results: Records pass/fail status of test cases.

SDLC Models

Waterfall Model: A sequential SDLC model where each phase is completed before the next begins.

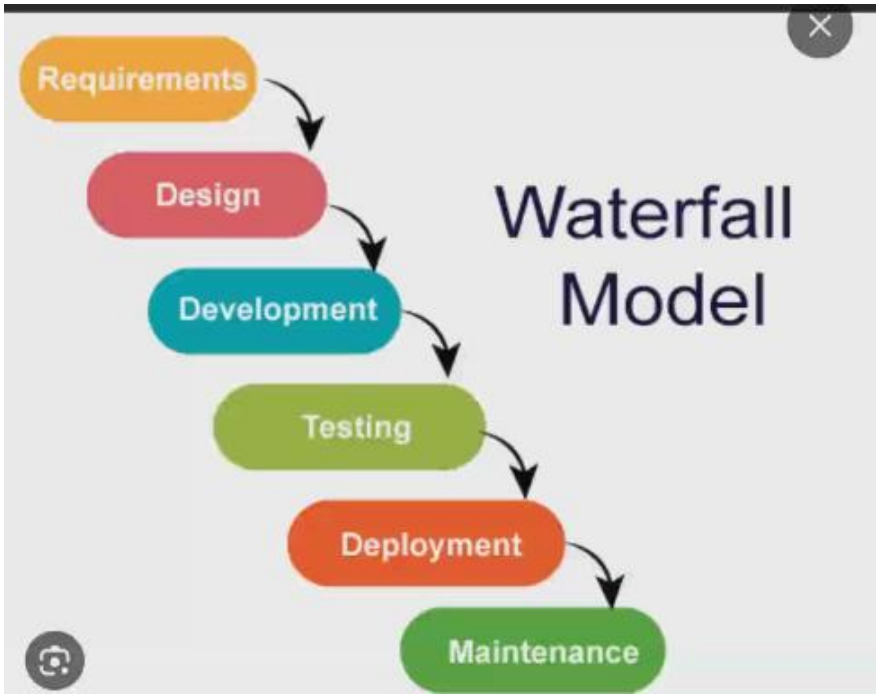
Agile Methodology: An iterative SDLC approach focused on flexibility, collaboration, and continuous improvement.

Agile Roles

Product Owner: Defines product requirements and prioritizes the product backlog.

Scrum Master: Facilitates Agile processes and removes team impediments.

Development Team: Designs, develops, tests, and delivers product increments.



Day 5

Python: A high-level programming language used to build backend and AI applications.

Backend: Server-side logic that processes requests and sends responses.

API: Application Programming Interface used for communication between client and server.

Flask: A lightweight Python framework used to build simple backend APIs.

FastAPI: A modern Python framework used to build high-performance APIs.

Route: A URL path that maps a request to a backend function.

Request: Data sent from client to backend.

Response: Data sent from backend to client.

JSON: A data format used to exchange data between client and server.

HTTP GET: A method used to fetch data from the server.

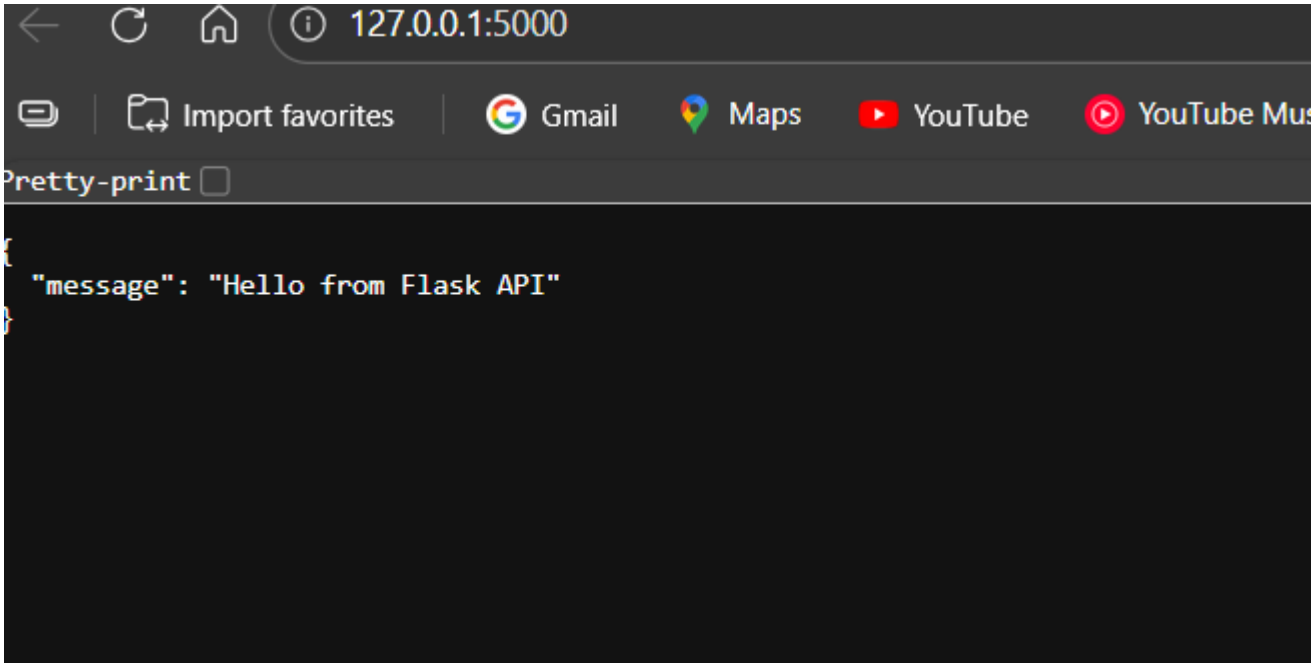
Basic Flask Code Example

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return jsonify({'message': 'Hello from Flask API'})

if __name__ == '__main__':
    app.run(debug=True)
```



Day -6

1. SQL Join Operations

SQL JOIN operations are used to combine rows from two or more tables based on a related column between them.

1.1 INNER JOIN

INNER JOIN returns only the matching records from both tables.
Example:

```
SELECT users.name, orders.amount
FROM users
INNER JOIN orders ON users.id = orders.user_id;
```

1.2 LEFT JOIN

LEFT JOIN returns all records from the left table and matching records from the right table. If there is no match, NULL values are returned for columns from the right table.
Example:

```
SELECT users.name, orders.amount
FROM users
LEFT JOIN orders ON users.id = orders.user_id;
```

1.3 RIGHT JOIN

RIGHT JOIN returns all records from the right table and matching records from the left table.
Example:

```
SELECT users.name, orders.amount
FROM users
RIGHT JOIN orders ON users.id = orders.user_id;
```

1.4 FULL JOIN

FULL JOIN returns all records when there is a match in either left or right table.
Example:

```
SELECT users.name, orders.amount
FROM users
FULL JOIN orders ON users.id = orders.user_id;
```

1.5 Basic SQL Operations

Other fundamental SQL operations include:
- SELECT: Retrieve data from a table.
- INSERT: Add new records.
- UPDATE: Modify existing records.
- DELETE: Remove records.
- WHERE: Filter records based on conditions.

Day- 7

2. Introduction to JWT (JSON Web Token)

JWT (JSON Web Token) is a secure method for transmitting information between parties as a JSON object. It is commonly used for authentication and authorization in web applications.

2.1 Structure of JWT

A JWT consists of three parts:
1. Header – Contains the token type and signing algorithm.
2. Payload – Contains user data and claims.
3. Signature – Used to verify the token's authenticity.

2.2 Creating a JWT in Python

Steps to implement JWT using PyJWT:
1. Install the library: pip install PyJWT
2. Import required modules.
3. Create a secret key.
4. Define a payload with user details.
5. Set expiration time using datetime.

6. Encode the token.

Example Code:

```
import jwt
import datetime

secret_key = 'my_secret_key'

payload = {
    'user_id': 1,
    'email': 'user@example.com',
    'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
}

token = jwt.encode(payload, secret_key, algorithm='HS256')
```

2.3 Benefits of JWT

- Stateless authentication.
- Secure token verification.

Day - 8

Hashing & Password Security

Hashing: Converts data into a fixed-length value using a mathematical function. Example: hash('abc')

Password Hashing: Converts a password into an unreadable value for secure storage. Example: bcrypt.hashpw(b'1234', bcrypt.gensalt())

Hashed passwords cannot be converted back to original form (one-way process).

Encoding: Reversible process to convert data format. Example: 'abc'.encode()

Encryption: Reversible process using a key to secure data.

Bcrypt: Secure algorithm designed for password hashing.

Salt: Random value added before hashing to improve security.

gensalt(): Generates a random salt in bcrypt. Example: bcrypt.gensalt()

bcrypt.hashpw(): Hashes password using bcrypt.

String to bytes: Required because bcrypt works with byte data. Example: b'password'

Example code:

```
import bcrypt

password = "1234"

# Step 1: Convert string to bytes

password_bytes = password.encode('utf-8')
```

```
# Step 2: Generate salt

salt = bcrypt.gensalt()

# Step 3: Hash the password using salt

hashed_password = bcrypt.hashpw(password_bytes, salt)

# Step 4: Print results

print("Original Password:", password)

print("Generated Salt:", salt)

print("Hashed Password:", hashed_password)
```

output:

```
Original Password: 1234

Generated Salt: b'$2b$12$e9kZ2YkP...'

Hashed Password: b'$2b$12$e9kZ2YkP...x8YHjK3FqR5LzP'
```

String Built-in Methods (One Line + Input/Output)

strip(): Removes spaces. Input: ' hi ' → Output: 'hi'

join(): Joins list elements. Input: ['I','like'] → Output: 'I like'

replace(): Replaces substring. Input: 'hello' replace 'h' with 'y' → Output: 'yello'

startswith(): Checks start. Input: 'hello'.startswith('he') → Output: True

endswith(): Checks end. Input: 'hello'.endswith('lo') → Output: True

find(): Finds index. Input: 'hello'.find('e') → Output: 1

isdigit(): Checks digits. Input: '123' → Output: True

isalpha(): Checks alphabets. Input: 'abc' → Output: True

Dictionary Methods (One Line + Input/Output)

get(): Returns value. Input: {'name':'Tejal'}.get('name') → Output: 'Tejal'

keys(): Returns keys. Input: {'a':1,'b':2}.keys() → Output: dict_keys(['a','b'])

values(): Returns values. Input: {'a':1,'b':2}.values() → Output: dict_values([1,2])

items(): Returns pairs. Input: {'a':1}.items() → Output: dict_items([('a',1)])

update(): Updates dictionary. Input: d.update({'age':20}) → Output: Updated dictionary

pop(): Removes key. Input: {'a':1}.pop('a') → Output: 1

popitem(): Removes last item. Input: {'a':1,'b':2}.popitem() → Output: ('b',2)

clear(): Removes all items. Input: {'a':1}.clear() → Output: {}

Set Methods & Operations (One Line + Input/Output)

add(): Adds element. Input: {1,2}.add(3) → Output: {1,2,3}

remove(): Removes element. Input: {1,2,3}.remove(3) → Output: {1,2}

discard(): Removes safely. Input: {1,2}.discard(3) → Output: {1,2}

pop(): Removes random element. Input: {1,2}.pop() → Output: 1 or 2

clear(): Removes all. Input: {1,2}.clear() → Output: set()

union(): Combines sets. Input: {1,2}.union({2,3}) → Output: {1,2,3}

intersection(): Common elements. Input: {1,2}.intersection({2,3}) → Output: {2}

difference(): Unique elements. Input: {1,2}.difference({2,3}) → Output: {1}

Day -9

Preparing for milestone

Day -10

Presentation

Day -11

Presentation

Day -12

Presentation

Day -13

-File Handling Functions

1.open() – Opens a file.
Example: f = open("test.txt","w")

2.read() – Reads entire file content.
Example: print(open("test.txt").read())

3.readline() – Reads one line at a time.
Example: print(open("test.txt").readline())

4.readlines() – Reads all lines as a list.
Example: print(open("test.txt").readlines())

5.write() – Writes data into a file.
Example: f.write("Hello")

6.writelines() – Writes multiple lines into a file.
Example: f.writelines(["A\n","B\n"])

7.close() – Closes the file.
Example: f.close()

-Basic Built-in Functions

1.len() – Returns the length of an object.
Example: print(len("Python"))
Output:6

2.range() – Generates a sequence of numbers.
Example: print(list(range(1,4)))
Output:[1, 2, 3]

3.print() – Displays output on the screen.
Example: print("Hello")
Output:Hello

4.type() – Returns the datatype of a value.
Example: print(type(10))
Output:<class 'int'>

5.id() – Returns unique memory address of object.
Example: print(id(10))
Output: Memory address (integer value)

6.sorted() – Returns a sorted list.
Example: print(sorted([3,1,2]))
Output: [1, 2, 3]

7.enumerate() – Returns index and value pairs.
Example: print(list(enumerate(["a","b"])))
Output:[(0, 'a'), (1, 'b')]

8.zip() – Combines multiple iterables into pairs.
Example: print(list(zip([1,2],[3,4])))
Output:[(1, 3), (2, 4)]

-Conversion Functions

int() – Converts value into an integer.
Example: print(int("10"))
Output:10

float() – Converts value into decimal number.
Example: print(float("5.5"))
Output:5.5

str() – Converts value into string.
Example: print(str(100))
Output:100

list() – Converts collection into list.
Example: print(list((1,2,3)))
Output:[1, 2, 3]

dict() – Converts paired data into dictionary.
Example: print(dict([(1,"a"),(2,"b")]))
Output:{1: 'a', 2: 'b'}

set() – Converts data into set (unique values).
Example: print(set([1,1,2]))
Output:{1, 2}

tuple() – Converts collection into tuple.
Example: print(tuple([1,2,3]))
Output:(1, 2, 3)

-Mathematical Functions

abs() – Returns positive value of number.
Example: print(abs(-10))
Output: 10

sum() – Adds all values in list/tuple.
Example: print(sum([1,2,3]))
Output: 6

min() – Finds the smallest value.
Example:print(min(5,2,9))
Output:2

max() – Finds the largest value.
Example: print(max(5,2,9))
Output: 9

pow() – Returns power of a number.
Example: print(pow(2,3))
Output: 8

round() – Rounds number to nearest value.
Example: print(round(4.6))
Output: 5

-Class & Object Related Functions

getattr() – Gets value of object's attribute.
Example:

```
class A:
 name="Tejal"
print(getattr(A,"name"))
```

Output: Tejal

setattr() – Sets or updates attribute value.
Example:

```
class A: pass
setattr(A,"age",20)
print(A.age)
```

Output: 20

hasattr() – Checks if attribute exists.
Example:

```
class A:
 name="Tejal"
print(hasattr(A,"name"))
```

Output: True

delattr() – Deletes an attribute.
Example:

```
class A:
 age=20
delattr(A,"age")
print(hasattr(A,"age"))
```

Output: False

isinstance() – Checks if object belongs to class/type.
Example: print(isinstance(10,int))
Output: True

issubclass() – Checks if class is subclass of another class.:

```
class A: pass
class B(A): pass
print(issubclass(B,A))
```

Output: True

-Exception Handling

try-except-finally – Handles errors safely without stopping program.
Example:

```
try:
 x=10/0
except:
 print("Error")
finally:
 print("Done")
```

Output: Error
Done

-Memory & Execution

del – Deletes variable or object from memory.
Example:

```
x=5
del x
```

Output: Variable deleted

gc.collect() – Forces garbage collection to free memory.
Example:

```
import gc
gc.collect()
```

Output: Garbage collection executed

eval() – Evaluates and runs a single expression.
Example:

```
print(eval("10+5"))
```

Output:15

exec() – Executes block of Python code.
Example: exec("a=5\nb=6\nprint(a+b)")
Output: 11

Day -14

AI Models: A program trained using data to recognize patterns and make decisions.

Google – Gemini Models: Large AI models developed by Google for multimodal reasoning and generation tasks.

OpenAI – ChatGPT: Conversational AI models used for text generation, reasoning, and coding assistance.

Microsoft – Copilot & Azure OpenAI: AI tools integrated into Microsoft products and cloud services for productivity and enterprise AI.

Groq: Provides ultra-fast inference hardware optimized for running LLMs efficiently.

NLP (Natural Language Processing): A field of AI that focuses on understanding and processing human language.

LLM (Large Language Model): A type of NLP model trained on massive datasets to understand and generate human-like text.

Google AI Studio: A platform used to test and build applications using Google Gemini AI models.

Rate Limit: Defines how many API requests can be sent to a server within a specific time period.

Token Limit: Counts both input and output text tokens processed by an AI model.

OpenAI API Example Code

```
pip install openai

import openai

response = client.chat.completions.create(
   model="gpt-4o-mini",
   messages=messages,
   temperature=0.7,
   max_tokens=500
)
```

Day- 15

NLP (Natural Language Processing)

NLP is a field of Artificial Intelligence that helps machines understand, interpret, and process human language.

NLTK (Natural Language Toolkit)

NLTK is a Python library used for NLP tasks such as tokenization, stopword removal, stemming, lemmatization, POS tagging, Named Entity Recognition (NER), and frequency distribution.

NLTK Key Steps

Tokenization – Breaks paragraph into words or sentences.

Stopword Removal – Removes common words like is, the, and.

Stemming – Reduces words to base form (running → run).

Lemmatization – Converts word into meaningful root form (better → good).

POS Tagging – Identifies grammatical parts of speech.

NER – Identifes named entities like person or location.

Frequency Distribution – Finds most common words

Day - 16

Machine Learning (ML): It is a subset of Artificial Intelligence that learns patterns from data and datasets to make predictions or decisions.

Types of Machine Learning

Supervised Learning: Uses labelled data where both input and output are known.

Regression: Used for continuous value prediction such as price or temperature.

Classification: Used when output is a category or label.

Unsupervised Learning: Learns patterns from unlabelled data.

Clustering: Groups data with similar characteristics (Example: customer grouping).

Reinforcement Learning: Learns through trial and error using rewards and penalties (Example: training a robot).

Embeddings

Embedding: Conversion of text, images, or words into numerical vectors so that computers and AI models can understand them easily.

Use Cases: Recommendation systems, search engines, similarity matching.

Logistic Regression

Logistic Regression is a supervised machine learning classification algorithm used to predict binary outcomes such as 0 or 1, Yes or No, True or False.

It uses the Sigmoid function to convert output into probability between 0 and 1.

Final Prediction Rule: If $y >= 0.5 \rightarrow$ Class = 1, If $y < 0.5 \rightarrow$ Class = 0.

Scikit-Learn (sklearn)

Scikit-learn is a Python machine learning library that provides tools for building ML models.

It supports classification, regression, clustering, and data preprocessing.

It helps build models easily without writing complex algorithms.

Example Import: from sklearn.linear_model import LogisticRegression

Decision Tree

Decision Tree is a supervised machine learning algorithm used for both classification and regression.

It makes decisions using a tree-like structure.

Root Node: Represents the starting point of the tree.

Internal Node: Represents decision conditions.

Leaf Node: Represents final